

PRODUCT REQUIREMENTS DOCUMENT

Dr Diet

“Eat What’s Right” — a healthy meal-subscription app for India

Version	Date	Status	Owner	Platform
2.4	29 Aug 2026	MVP built · backend live (Phase 1)	Dr Diet Product	Responsive web (Next.js)

Dr Diet lets busy professionals in India subscribe to nutritionally-balanced meals delivered on a schedule that fits their life, with minimal planning effort. This document defines the product: who it serves, the end-to-end flow, what is built today, and the requirements and roadmap to a fully live, operationally-automated service. It reflects the current implementation — a complete functional app with a customer storefront, an admin console, and a real Supabase backend — and adds forward requirements for authentication, flexible scheduling, and automated delivery logistics.

1 · Product summary & goals

What it is

A mobile-first web app where a user discovers meal plans, builds a personalised plan through a guided wizard (or picks a curated plan), checks out with serviceability-aware delivery, pays, and manages an ongoing subscription from a dashboard. An operations-facing admin console manages customers, subscriptions, orders, menu, coupons, payments, reviews and support.

Goals

- Take a first-time visitor from landing to a **confirmed, paid subscription with zero dead ends**.
- Remove meal-planning effort while keeping nutrition aligned to a fitness goal.
- Give customers **real scheduling control** — plan ahead or change day-to-day.
- **Automate fulfilment & protect margins** — hands-off delivery at an optimised cost.

Non-goals (this release)

AI/photo-based body analysis · referral & rewards mechanics · native mobile apps (responsive web first) · in-house rider fleet or route optimisation (we integrate aggregators, not build logistics) · production-grade payment settlement (simulated until Phase 2).

2 · Primary persona

Raj Mehta · 25 · Corporate engineer & gym-goer

Works 8 hrs/day, 5 days/week; gyms 4x/week. Conscious about food for fitness, but unwilling to spend time planning meals. **Trust bar is high, patience is low** — he distrusts new brands and abandons any step that feels like too much commitment before he’s ready.

Wants: get fit · meals matched to his goal and macros · spend less time planning · **flexibility to change or skip a day** · **a fast, password-free login**. **Fears:** unclear billing, taste risk, “do they even deliver to me?”, being locked in.

3 · Journey-driven requirements

Stage	Raj's pain point	Product answer (shipped)
Discover	Doesn't trust brands; hard to choose among many.	Home leads with rating/review count, FSSAI & dietitian trust badges, one-line value prop.
Explore	Comparing meals & nutrition is hard; distrusts nutrition facts.	Explore + side-by-side Compare with calories/protein/carbs/fat; Reviews & FAQs reachable without login.
Build plan	Doesn't know body stats; unsure of allergies; won't over-commit.	Skippable body-analysis step with a standard calorie target; explicit diet + allergy selection flagged on meal cards; a Trial path before full commitment.
Checkout	When do I start? Do they deliver here? Taste risk.	Serviceability (pincode) checked first, start-date picker with plain first-delivery date, chip + free-text delivery instructions.
Payment	Fear of failure; "too much?"; "does this auto-renew?"	Itemised ₹ breakdown, explicit billing-cadence copy , one-tap coupon, UPI-first methods, success/failure/retry states.

4 · Feature scope & status

Area	Capabilities	Status
Auth & login	One-time code (OTP) by email or phone; polished code entry (see §5). Delivery of codes needs providers	Partial
Discover	Home, Explore, Compare, Plan details, This-week's menu, Reviews, FAQs	Built
Plan builder	Goal → Body (skippable) → Diet → Allergies → Meals → Duration (incl. Trial) → Preferences → Customisation → Summary	Built
Checkout	Serviceability-first address, slot, instructions, start date, coupon, itemised review	Built
Payment	UPI-first method picker, processing, success / failure / retry, confirmation	Simulated
Dashboard	Today's meals, skip/pause, change time/address, upgrade, progress, invoices, support, profile	Built
Scheduling flexibility	Plans + skip + pause built; meal-level pre-select / daily edit / swap not yet (see §6)	Partial
Delivery & logistics	Automated aggregator assignment, tracking, POD, rate optimisation — not started (see §7)	Gap
Admin console	Dashboard/KPIs, orders, subscriptions, customers, menu, coupons, payments, reviews, support, settings	Built
Backend	Supabase Postgres, auth, Row-Level Security; profiles, subscriptions, orders, invoices persisted	Live

Built/Live shipped · **Simulated** functional, not real yet · **Gap** required, not started · **Later** planned.

5 · Authentication & login (requirements)

Password-free, one-time-code sign-in only — fast enough for a low-patience user, with no social logins and no guest checkout. Explore & Compare stay public; a verified account is required to build a plan and order.

Requirement	Detail	Status
OTP by email	Enter email → receive a 6-digit one-time code → verify.	Built
OTP by phone	Enter mobile (+91) → receive a 6-digit SMS code → verify.	Built
No passwords / social / guest	No password, Google, or guest login. Public browsing stays open; sign in to order.	Built
Polished code entry	Segmented 6-box input with auto-advance, paste, and auto-verify on the last digit.	Built
Resend with cooldown	“Resend code” with a 30-second countdown; change email/number; clear error & loading states.	Built
Auto-provisioned profile	A profile row is created per new user; each user accesses only their own data (RLS).	Built
Email code delivery	Custom SMTP (e.g. Resend) — built-in email can’t show a code. Production needs a verified sending domain .	Provider setup
Phone code delivery	An SMS provider (e.g. Twilio / MSG91) connected to Supabase Auth.	Provider setup

Note: the login experience is fully built; going live end-to-end only requires connecting the email (SMTP) and SMS providers above — no further app changes.

6 · Scheduling flexibility (requirements)

Customers must be able to plan ahead *or* stay fully day-to-day, without any change ever breaking the underlying subscription or billing. Orders mutate; the subscription and its cadence stay intact.

Requirement	Detail	Status
Weekly / monthly plans	Subscribe by duration (trial / weekly / monthly / quarterly).	Built
Skip a delivery	Skip/un-skip an individual day from the dashboard.	Built
Pause / resume	Pause the whole subscription and resume later.	Built
Pre-select meals ahead	Choose the specific dish for each slot across a week or month in advance.	Required
Edit meals daily	Change the dish for a given day up to a daily cut-off, for max flexibility.	Required
Swap meals	Substitute one dish for another (within diet/allergy rules) without disrupting the plan.	Required

System rule: weekly/monthly plans, daily changes, skips, pauses and swaps must all operate on the order layer while the subscription record and billing cadence remain unbroken. A cut-off time governs how late an edit is allowed before kitchen lock.

7 · Delivery & logistics (requirements)

7.1 · Automated third-party delivery assignment Required · not started

Orders must be auto-assigned to a third-party delivery partner via API — **no manual rider booking or follow-ups** from the team.



7.2 · Optimised delivery pricing to protect margins Required · not started

Delivery must not erode meal margins. Today it is a flat fee with no optimisation; the requirement is a multi-aggregator, rate-aware routing layer.

- **Multiple aggregators** — integrate several partners, no single locked-in provider.
- **Rate optimisation** — compute cost by distance, order volume, and time of day.
- **Cheapest-reliable routing** — route each order to the best-value dependable partner, per order.

Cost target: current ~₹120 for 7 km ≈ 48% of a ₹250 meal (unsustainable) → bring the effective 7 km cost to ₹60–80, depending on city and volume.

8 · Technical architecture

Stack

- **Next.js 16** (App Router) · **React 19** · **TypeScript**
- **Tailwind CSS v4** — CSS-first theming via `@theme` tokens
- Fonts: Barlow + Baloo 2 (accent stand-in) · Hosting: Vercel

The swappable data seam

All stateful operations go through a single **DrDietApi** interface. A `localStorage` mock is the zero-config default; the **Supabase adapter** **activates automatically** when environment keys are present — no UI code changes. The same seam is where delivery-partner and payment providers plug in.

Backend & security

- **Supabase Postgres**, schema mirrors the data model (INR-only, no beef).
- **Row-Level Security**: catalog world-readable; users access only their own data.
- **Auth**: one-time code (OTP) by email or phone only (see §5).

Planned integration surfaces

Messaging: SMTP (Resend) for email codes + verified domain; SMS provider for phone codes. **Payments**: Razorpay (order + webhook, UPI Autopay). **Delivery**: a routing service fanning out to multiple aggregator APIs, with webhooks for tracking & proof-of-delivery.

9 · Constraints (non-negotiable)

₹	0	1	0
INR ONLY — NO AED / USD / \$ ANYWHERE	BEEF REFERENCES — IN MENU, DATA OR COPY	FIXED BRAND SYSTEM — TOKENS, NO AD-HOC HEX	DEAD-END CTAS — EVERY ACTION NAVIGATES

10 · Success criteria & metrics

Acceptance

- Home → confirmed paid subscription with no dead-end click.
- A user signs in with an emailed/texted OTP and a purchase persists to the backend, visible across devices and in admin.
- A customer can pre-plan a week and also change/skip/swap a single day without breaking billing.
- An order auto-allocates a rider and captures proof of delivery with no manual step.

Product & ops metrics

- OTP request → verify success rate; time-to-sign-in
- Builder completion & checkout conversion; Trial → paid
- Payment success rate; involuntary churn from failed renewals
- Pause/skip/swap rate, MRR; **delivery cost per order** & % of meal price; on-time & POD rate

11 · Roadmap

Phase	Theme	Scope	Status
Phase 1	Real backend & login	Supabase Postgres + auth + RLS; persist profile, subscription, orders, invoices; OTP login by email + phone with polished code entry.	Done
Phase 2	Providers, payments & ops	Connect SMTP (email codes) + SMS (phone codes); Razorpay order + webhook (activate on confirmed payment); UPI Autopay; repoint admin to Supabase.	Next
Phase 3	Flexible scheduling	Meal-level control: pre-select a week/month, daily edit & swap with cut-off, on the order layer; scheduled renewals & rolling order generation; notifications.	Planned
Phase 4	Delivery & logistics	Multi-aggregator delivery integration; auto rider allocation; live tracking + proof-of-delivery; rate-optimised cheapest-reliable routing (7 km → ₹60–80).	Planned
Later	Growth	AI/photo body analysis; referral & rewards; audience landing pages; native wrappers.	Backlog

12 · Open items & risks

- **OTP delivery not live** — login UX is built, but email codes need custom SMTP (+ verified domain) and phone codes need an SMS provider before anyone can actually sign in on production (\$5).
- **Payments are simulated** — no money moves until Razorpay (Phase 2).
- **Meal-level flexibility missing** — pre-select / daily edit / swap not built; meals are auto-assigned today (\$6).
- **No delivery automation** — order status is set manually in admin; no aggregator API, tracking, or POD (\$7.1).
- **Delivery cost unoptimised** — flat fee, no multi-aggregator routing; ~48% of meal price today vs. ₹60–80 target (\$7.2).